



Дипломная работа на тему:
“Классификация произведений известных художников с помощью компьютерного зрения”

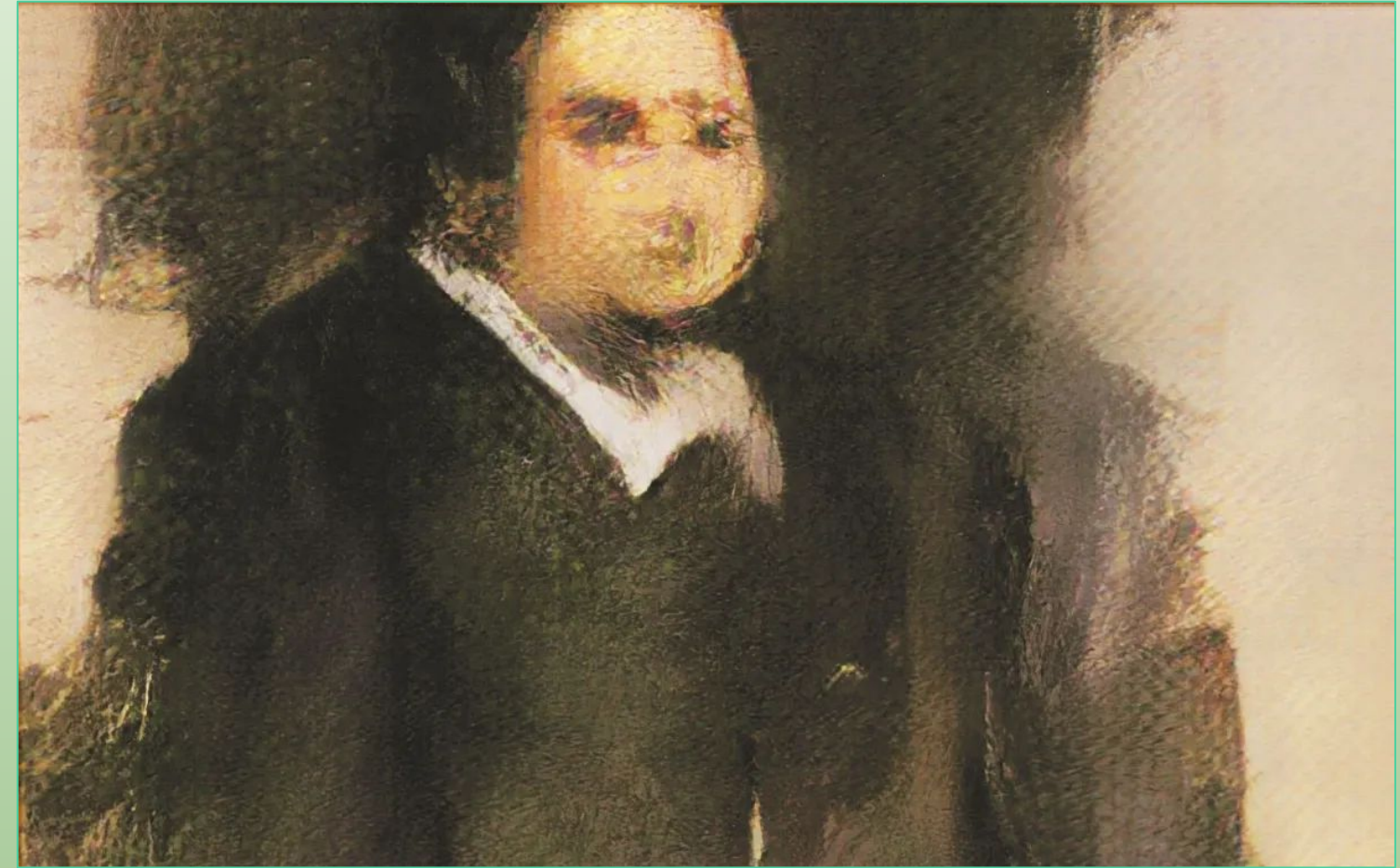


Автор: Загускина Юлия

Группа: DSU-22



Вводная часть



Древнейший из известных наскальных рисунков был обнаружен в пещере в Индонезии на юге острова Сулавеси, которому как минимум 45,5 тысяч лет, нанесён красной охрой (слева).

Картина «портрет Эдмонда Белами» , созданная искусственным интеллектом, ушла с молотка за \$0,5 млн. в 2018 году. Работа была создана французскими студентами (справа).



ПОСТАНОВКА ЗАДАЧИ

- Цель - научить модель предсказывать художника по картине, но не объекты на ней.
- Задача - обучить нейронную сеть на классификацию художников по их картинам с точностью на тестовой выборке выше 80%.
- Метрика качества: определить долю верно классифицированных объектов.



Анализ данных

Данные находятся в открытом доступе на платформе Kaggle, были собраны с сайтов wikipedia и artchallenge.ru.

Набор данных содержит 3 файла:

artists.csv - информация о каждом художнике (id, name, years, genre, nationality, bio, wikipedia, paintings), использовались для получения общего представления о данных. Были проверены на пропуски.

images.zip - коллекция изображений (в полном размере), разделенных по папкам и пронумерованных, использовались для обучения и тестирования. Тип данных: изображения в формате jpg.

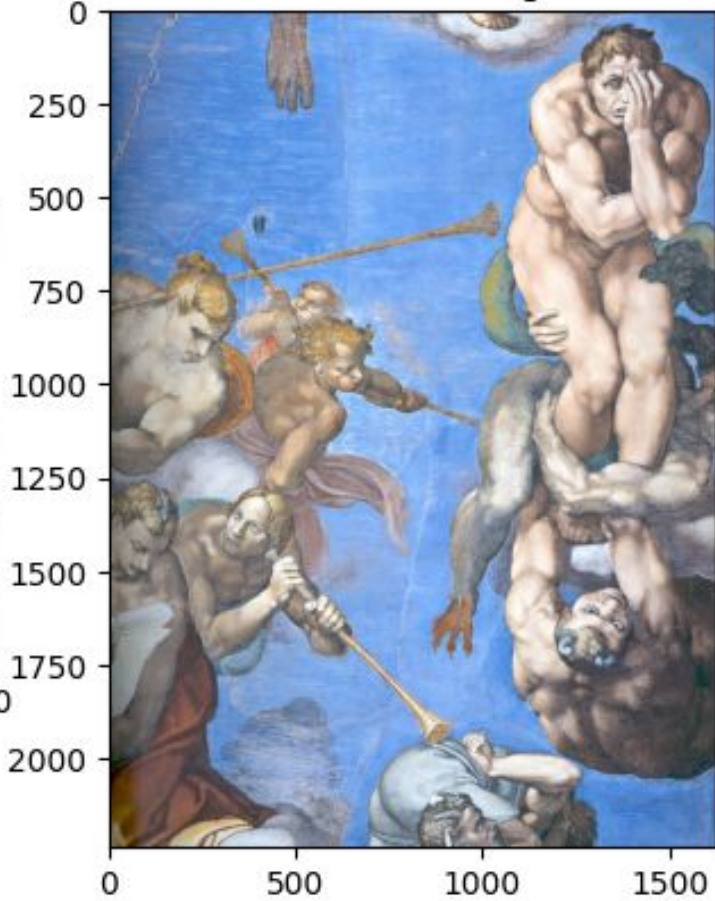
resized.zip - размеры изображений были уменьшены и извлечены из структуры папок.

Набор данных состоит из 8446 изображений картин разной размерности 50 известных художников.

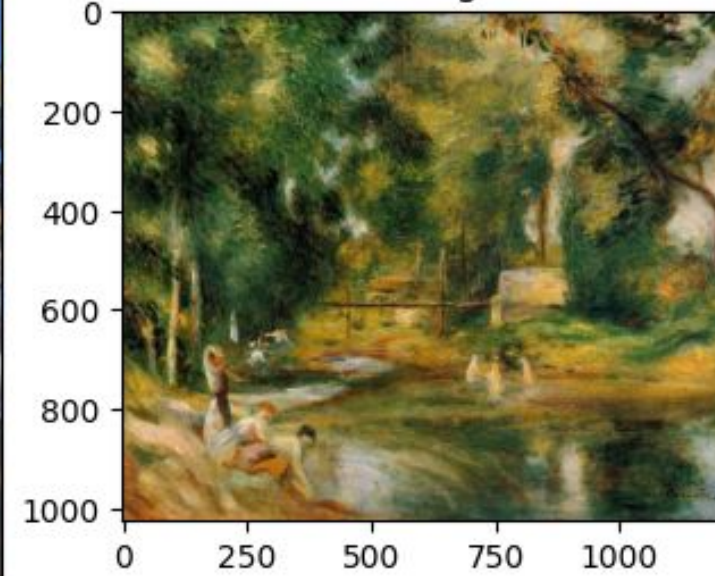


5 картин
случайных
художников

Artist: Michelangelo



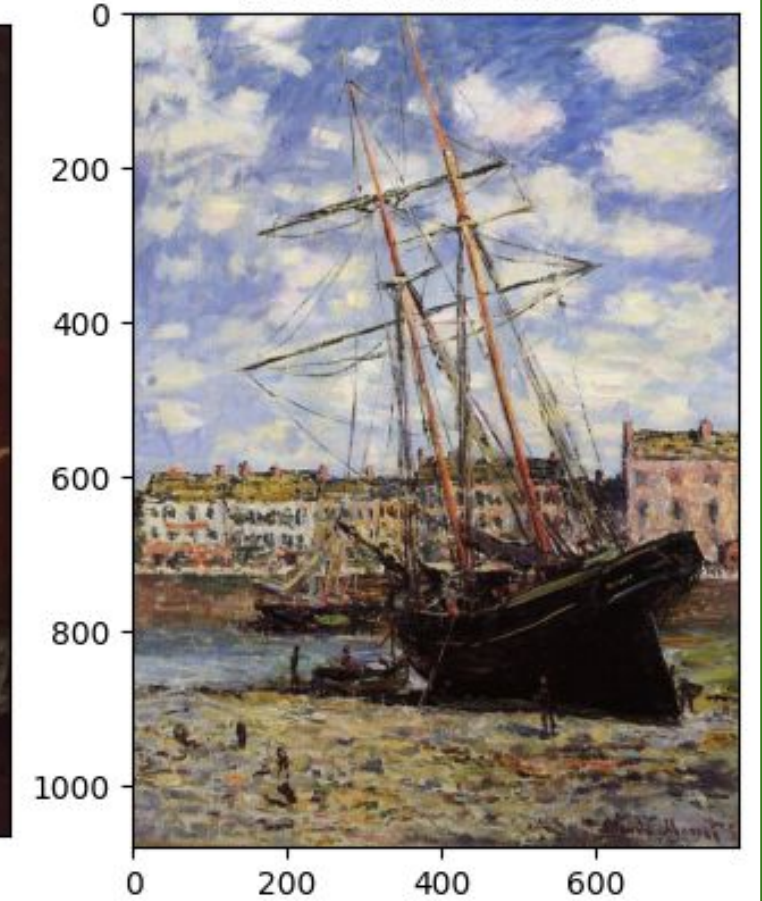
Artist: Pierre-Auguste Renoir



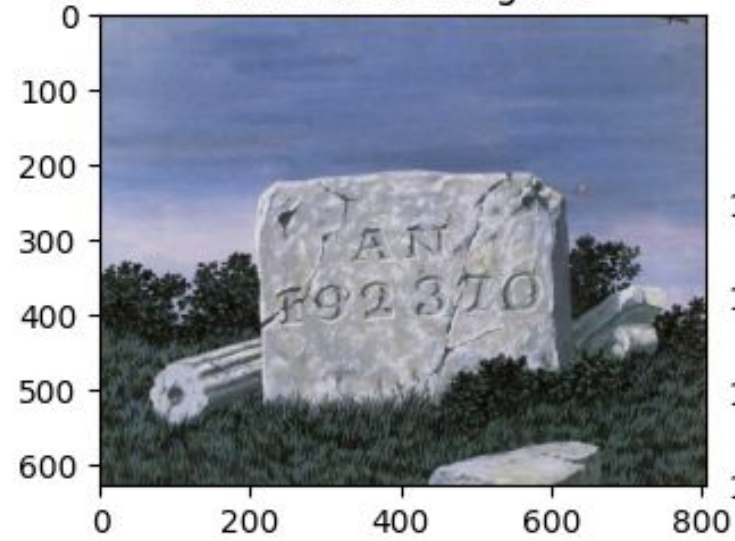
Artist: Gustave Courbet



Artist: Claude Monet

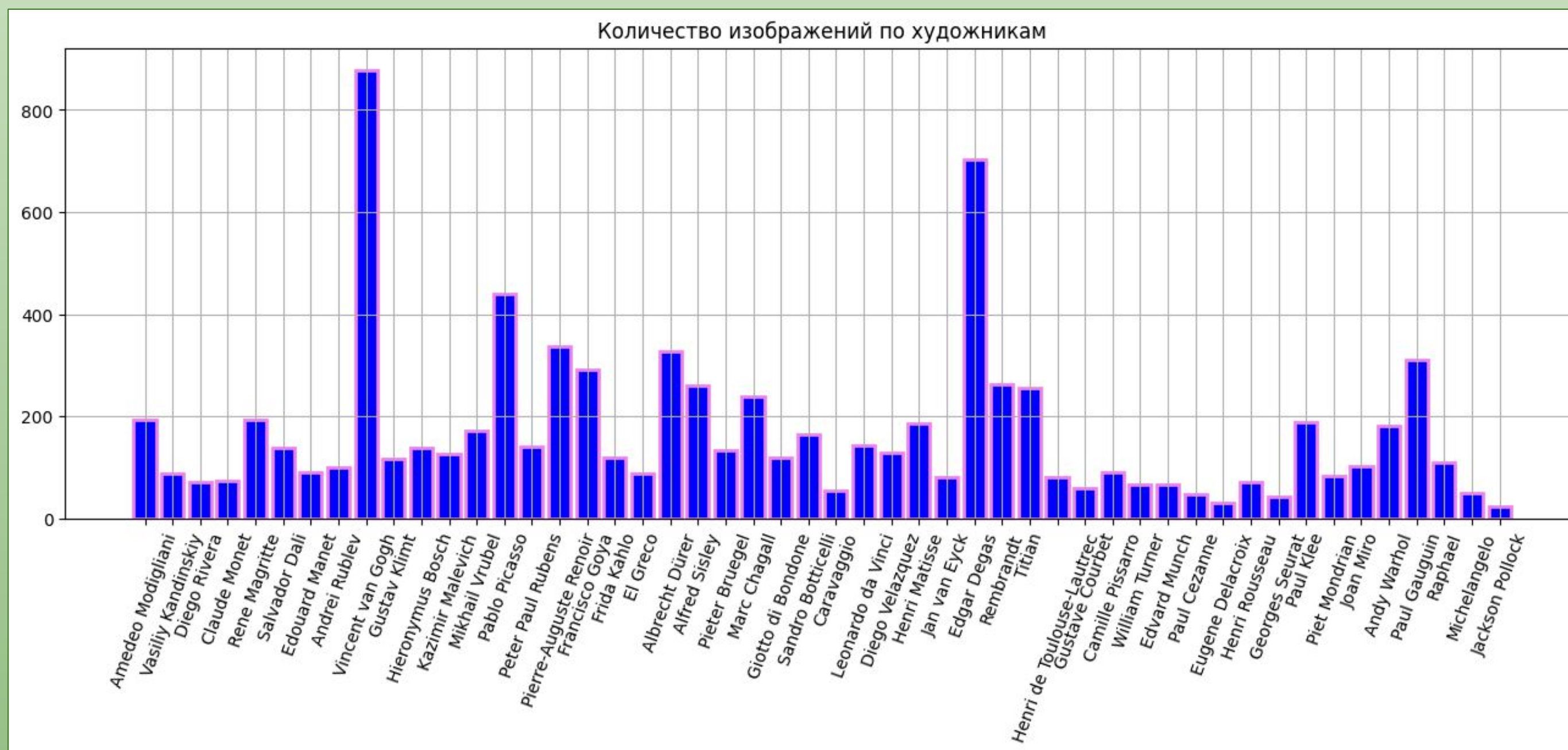


Artist: Rene Magritte

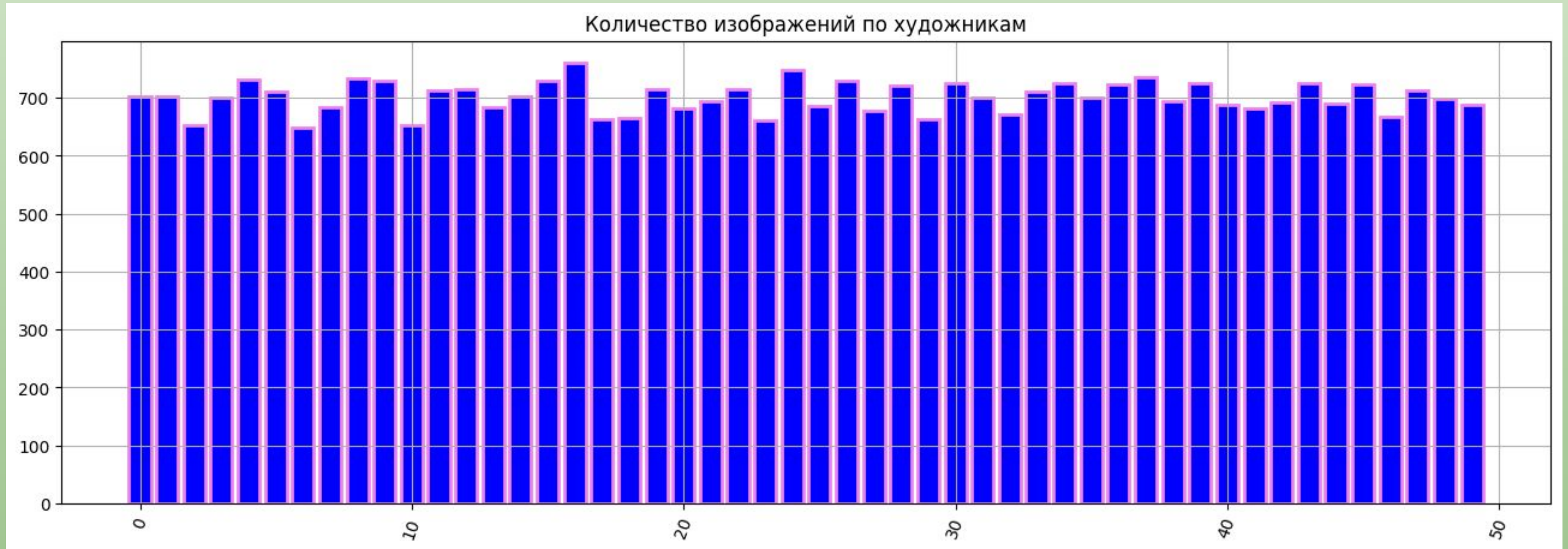


В данных имеется несбалансированность классов: 877 картин Ван Гога против 24 картины Джексона Поллока. Значит, модель не выучит редкие классы и с большей вероятностью будет присваивать этим классам метки больших классов на тестовых данных.

💡 Решение: `WeightedRandomSampler` (модель будет видеть изображения каждого класса приблизительно одинаковое количество раз)



Результат применения WeightedRandomSampler



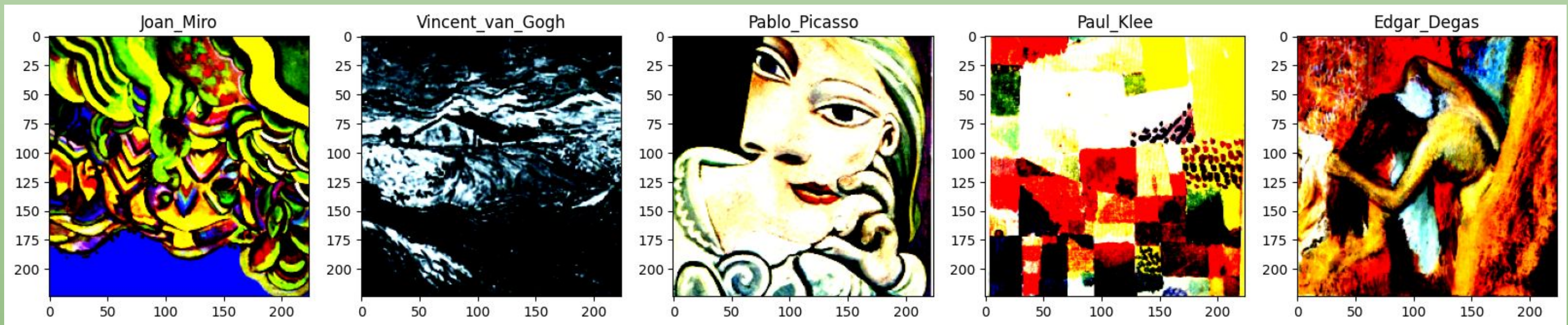
Подготовка изображений к обучению.

Тренировочная выборка - 80%, 6756 изображений.

Тестовая выборка - 20%, 1690 изображений.

К данным применяется нормализация (среднее, стандартное отклонение), также приведение данных к одному размеру 224*224.

К тренировочной выборке применяется аугментация: переворот изо по горизонтали (RandomHorizontalFlip), переворот изо по вертикали (RandomVerticalFlip), преобразование случайной перспективы (RandomPerspective).



Выбор архитектур

Модели будут обучаться на 20 эпохах на GPU, оптимизатор -Adam, шаг обучения - $1e-4$, функция потерь - cross-entropy.

1

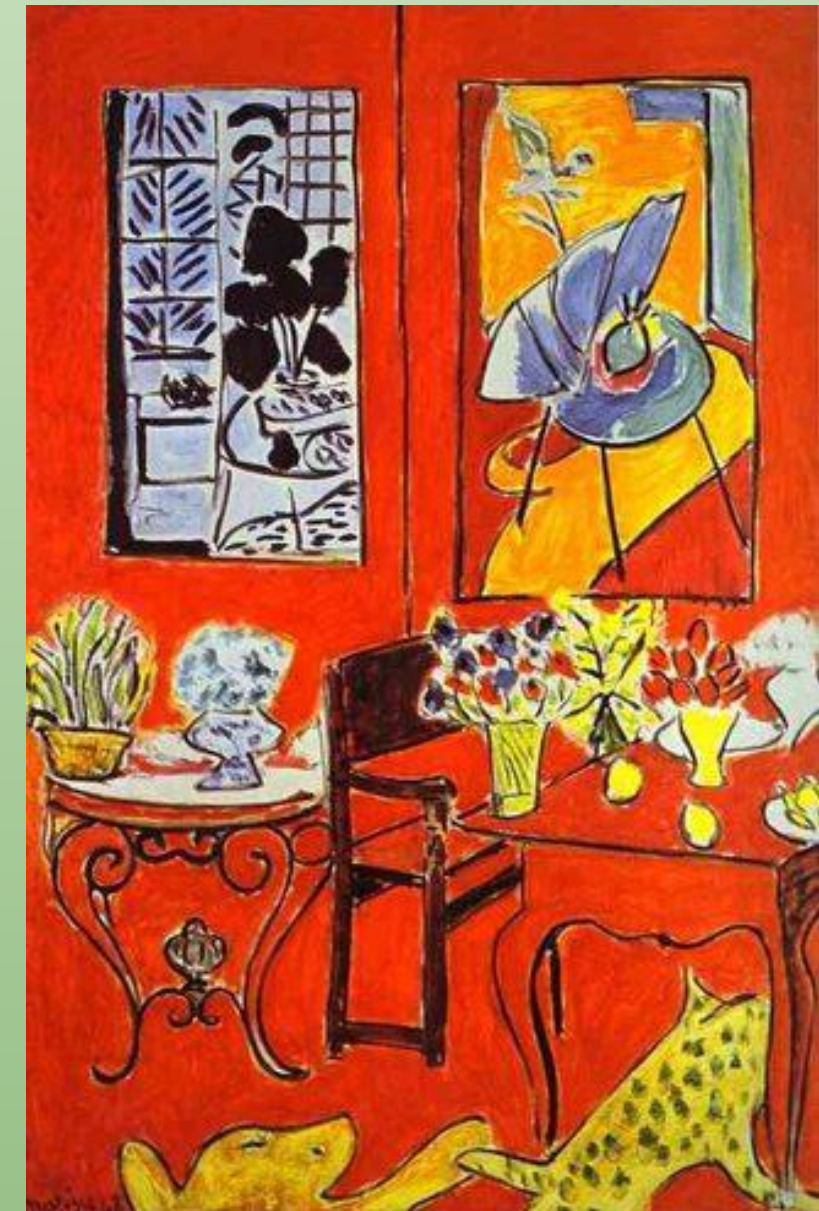
MobileNetv2

2

MobileNetv3

3

EfficientNetv2-small

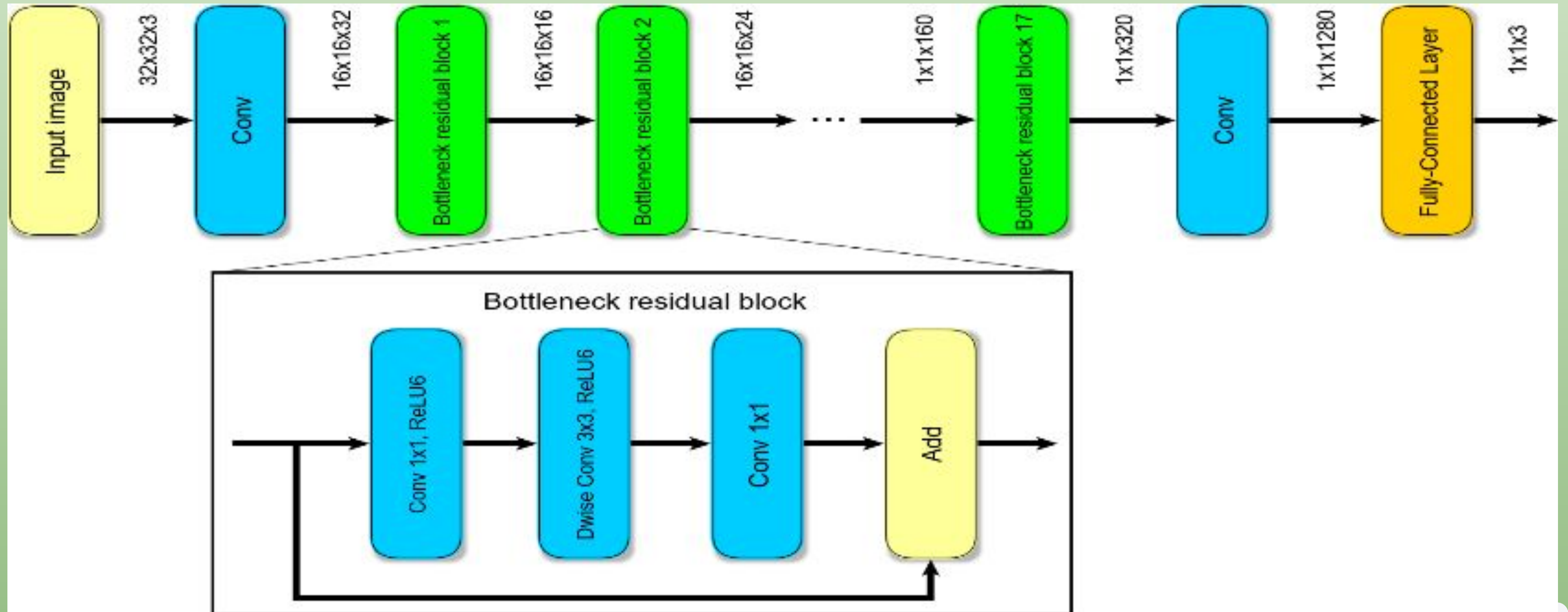


Г.Матисс



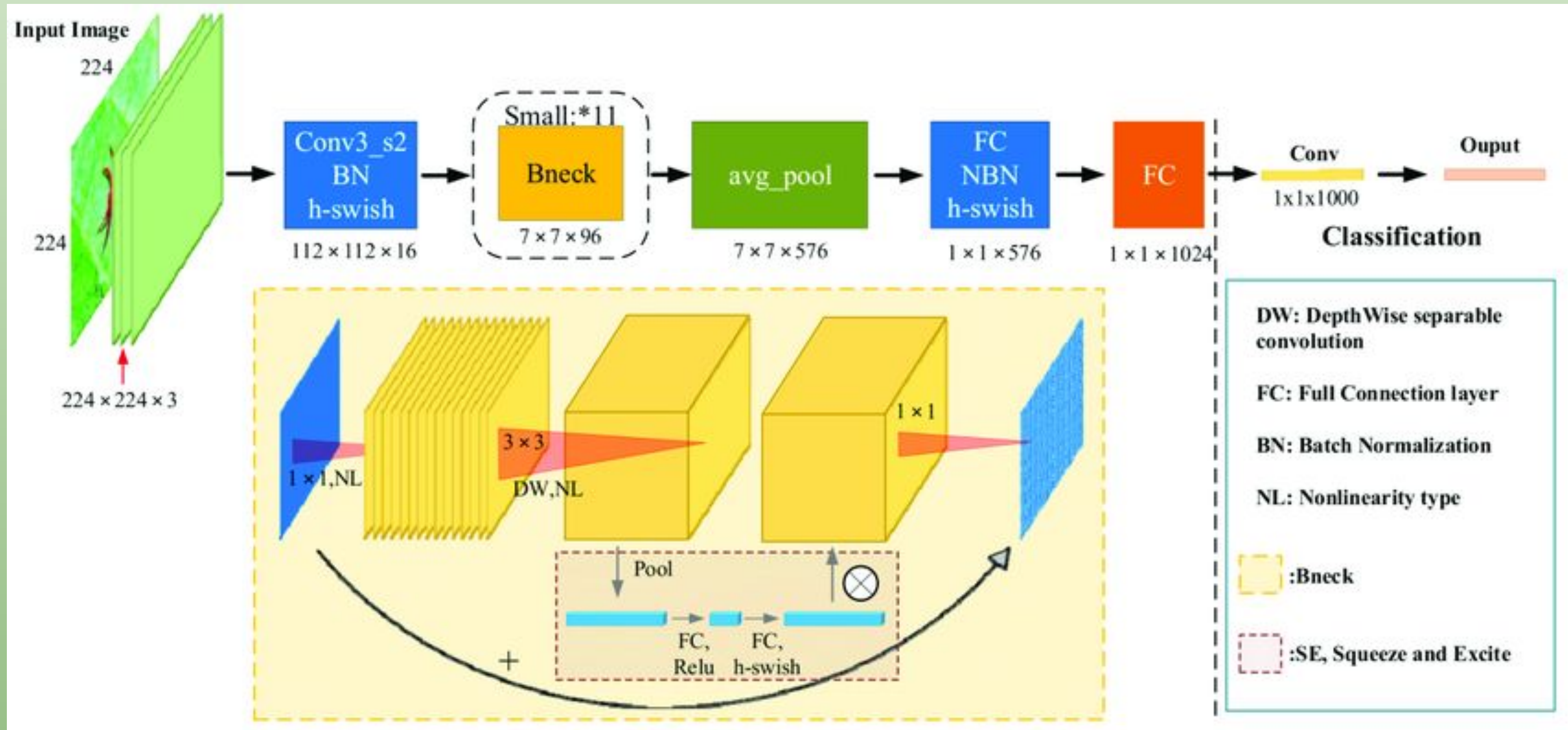
MobileNetv2

Архитектура предобучена на датасете ImageNet (1000 классов)



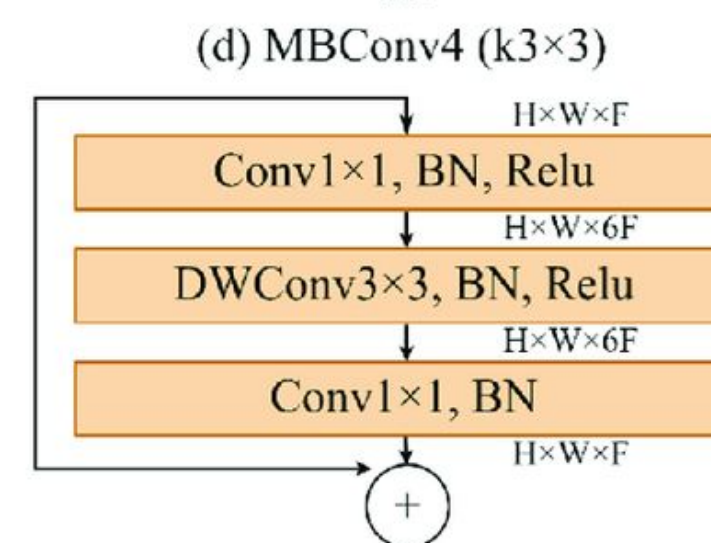
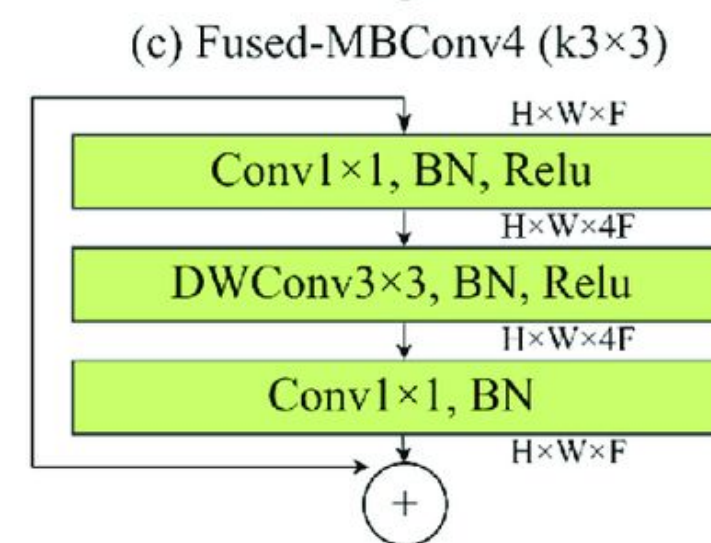
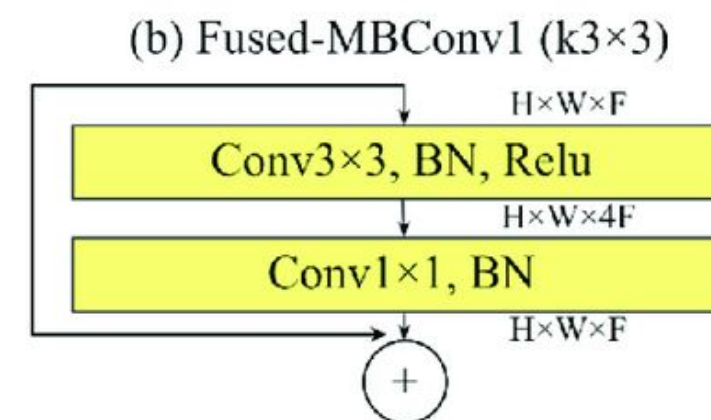
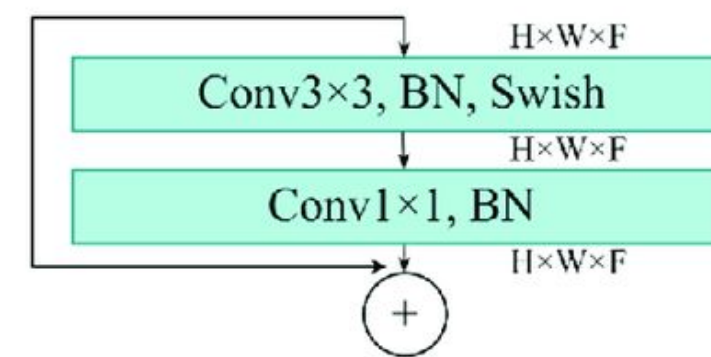
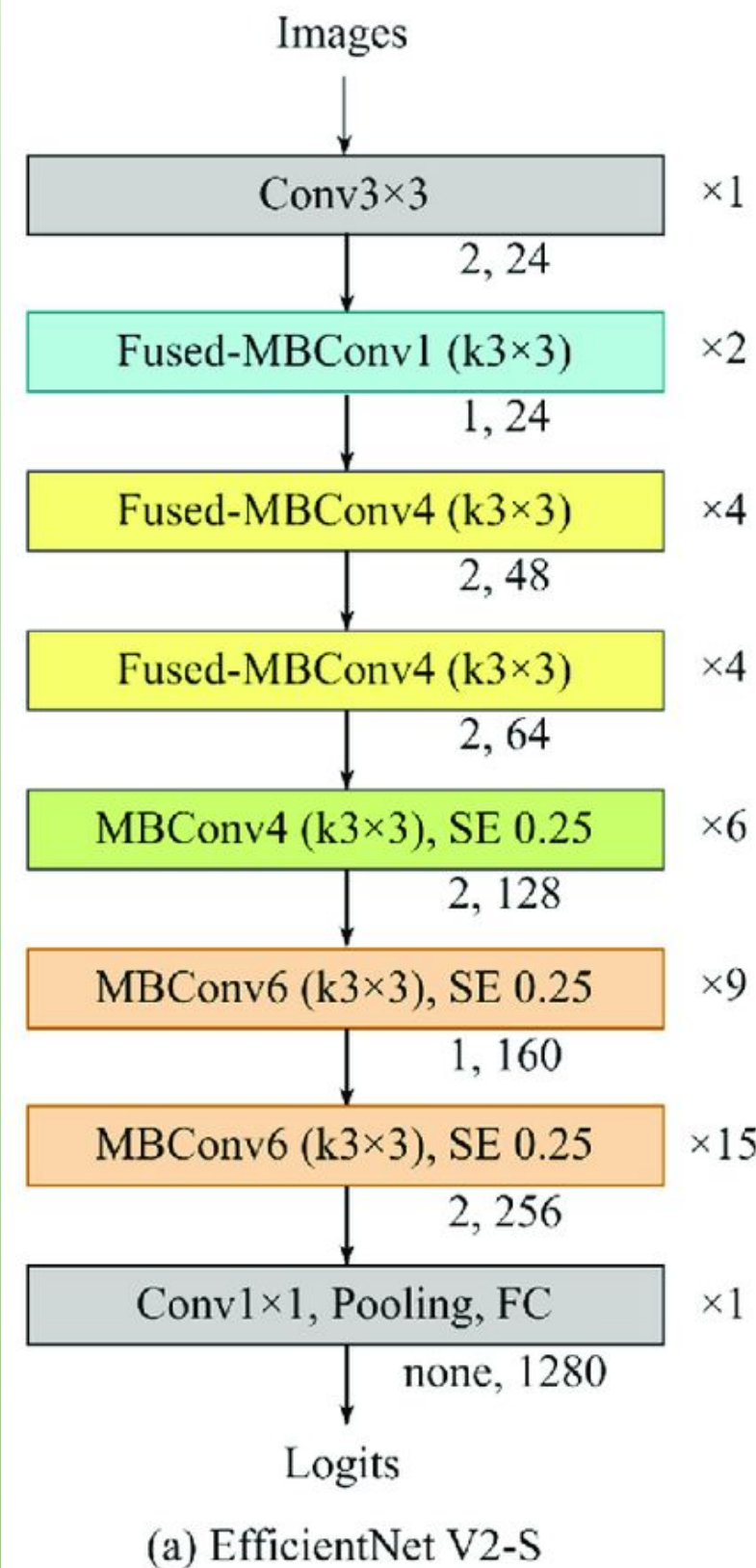
MobileNetv3 (добавлены блоки Squeeze-and-Excitation)

Архитектура предобучена на датасете ImageNet (1000 классов)



EfficientNetv2-small

Архитектура предобучена на датасете ImageNet21k (21841 класс)

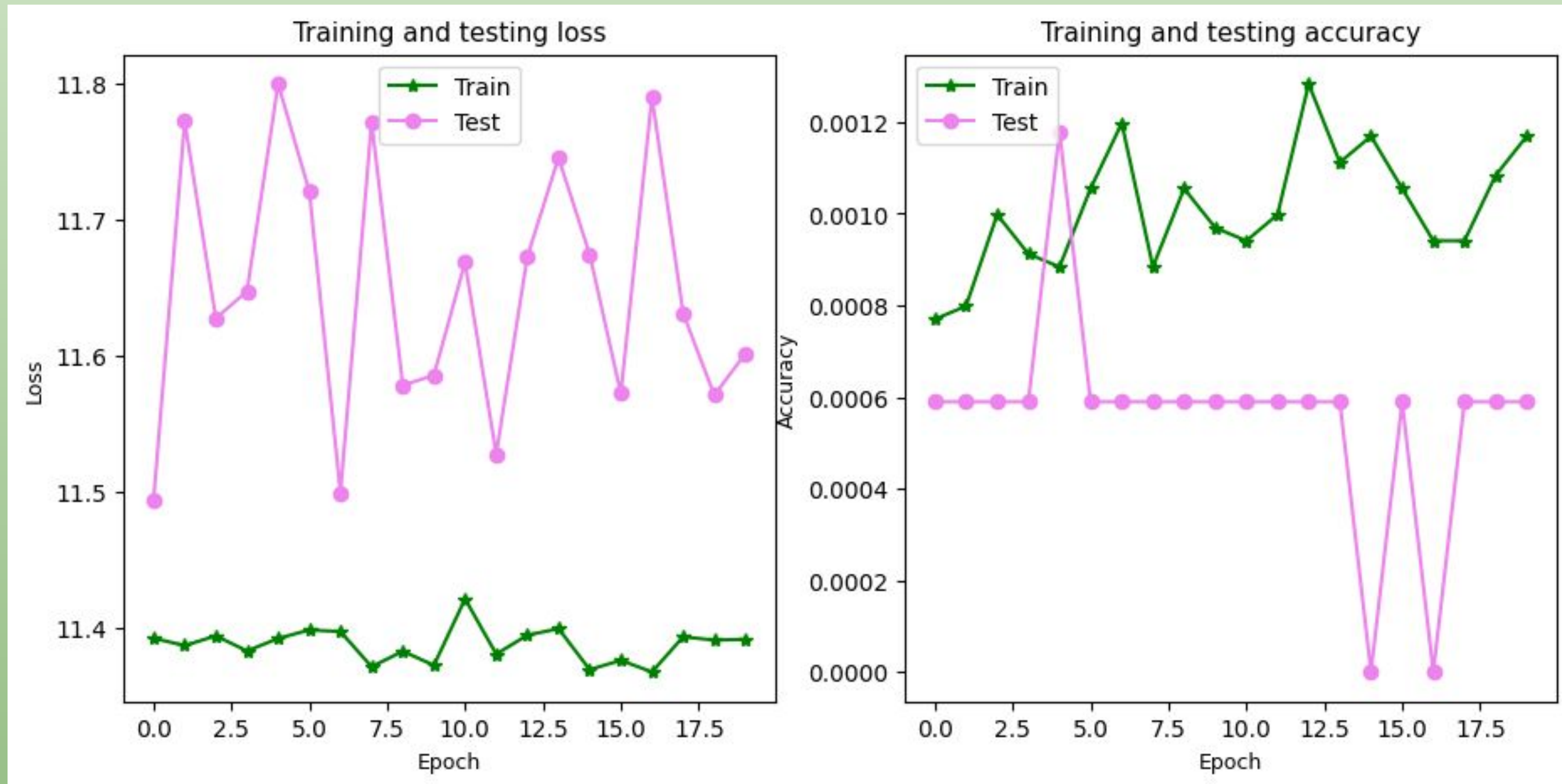


(e) MBConv6 (k3x3)

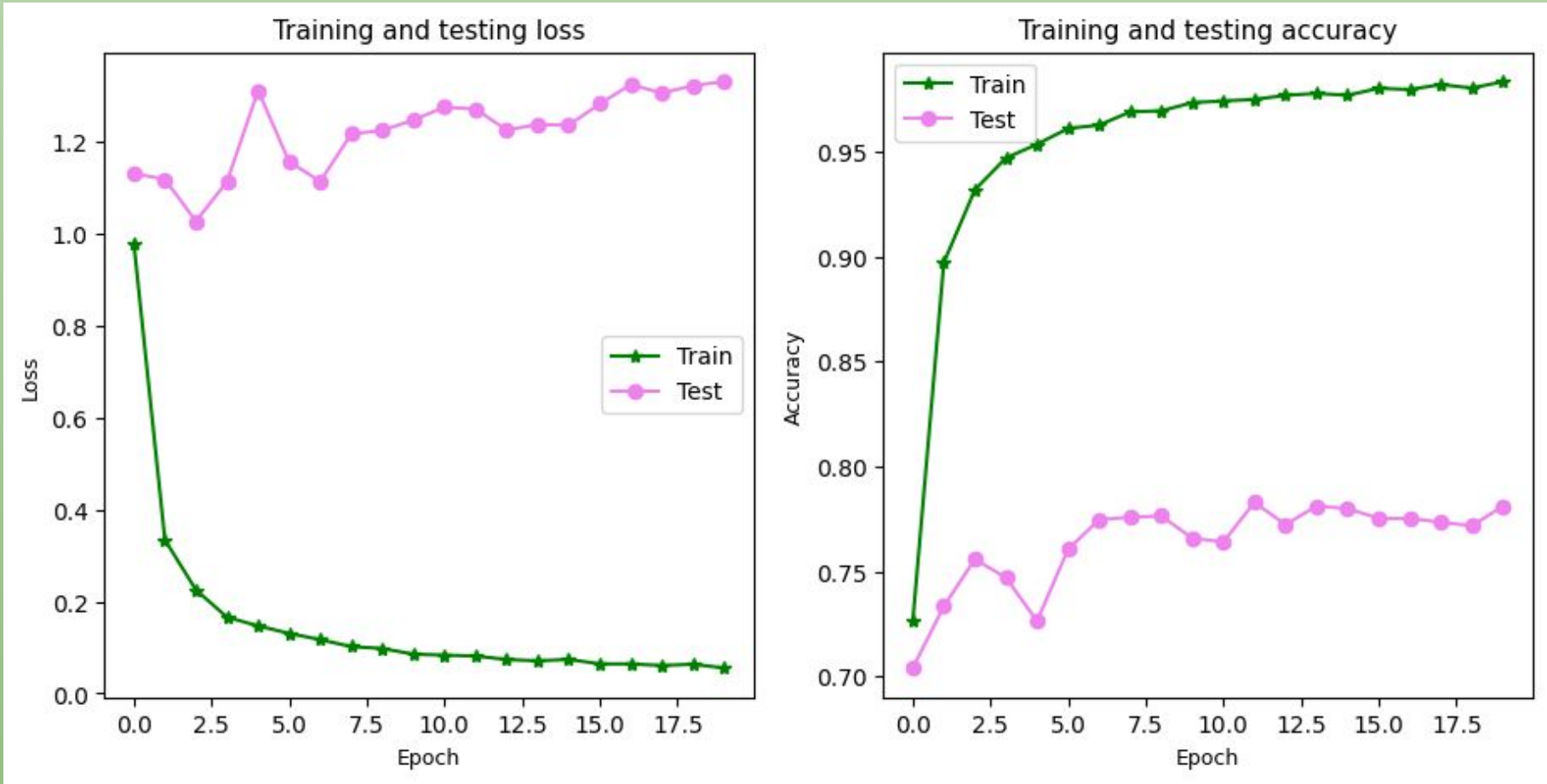
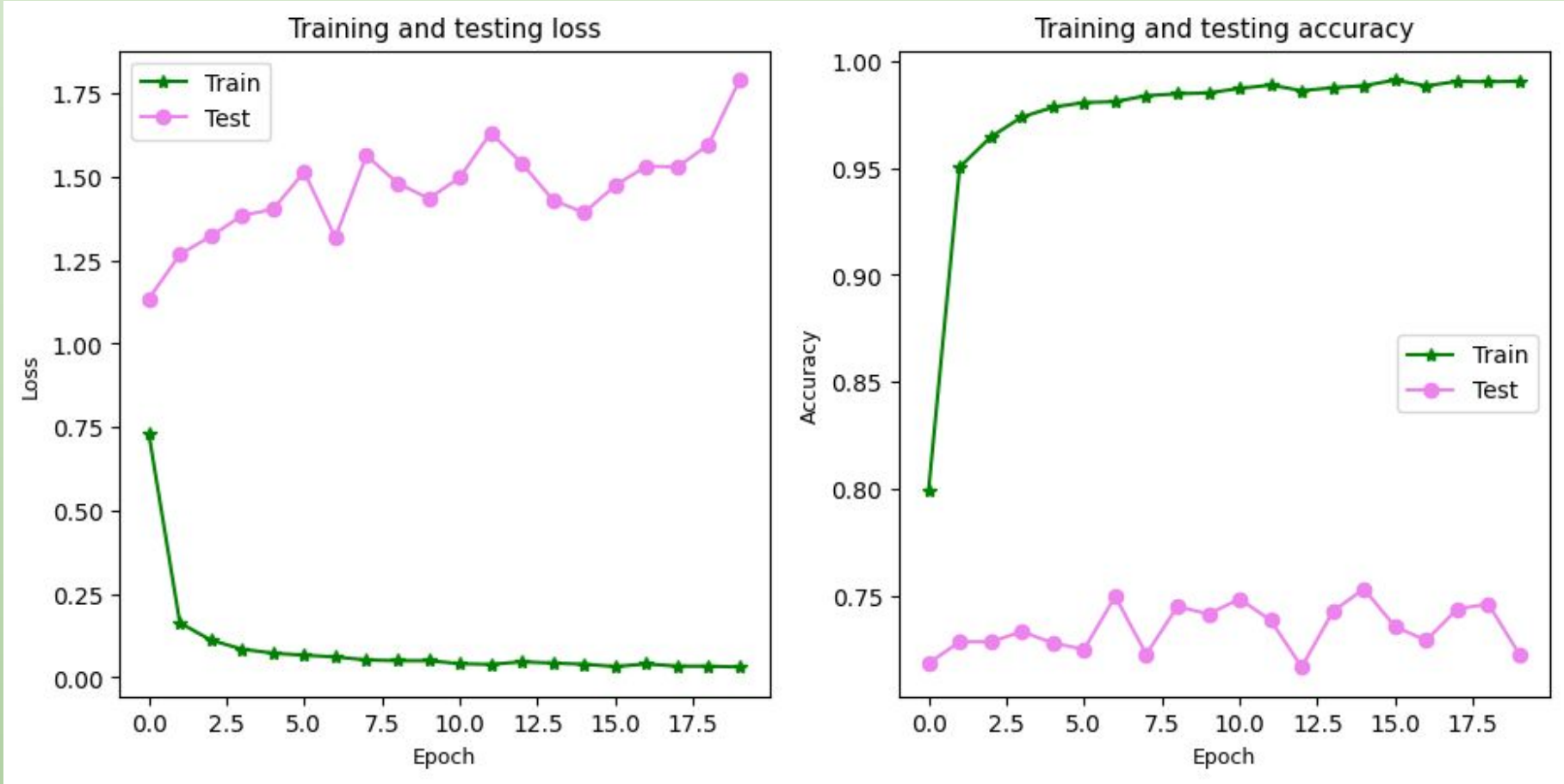


Проведённые эксперименты

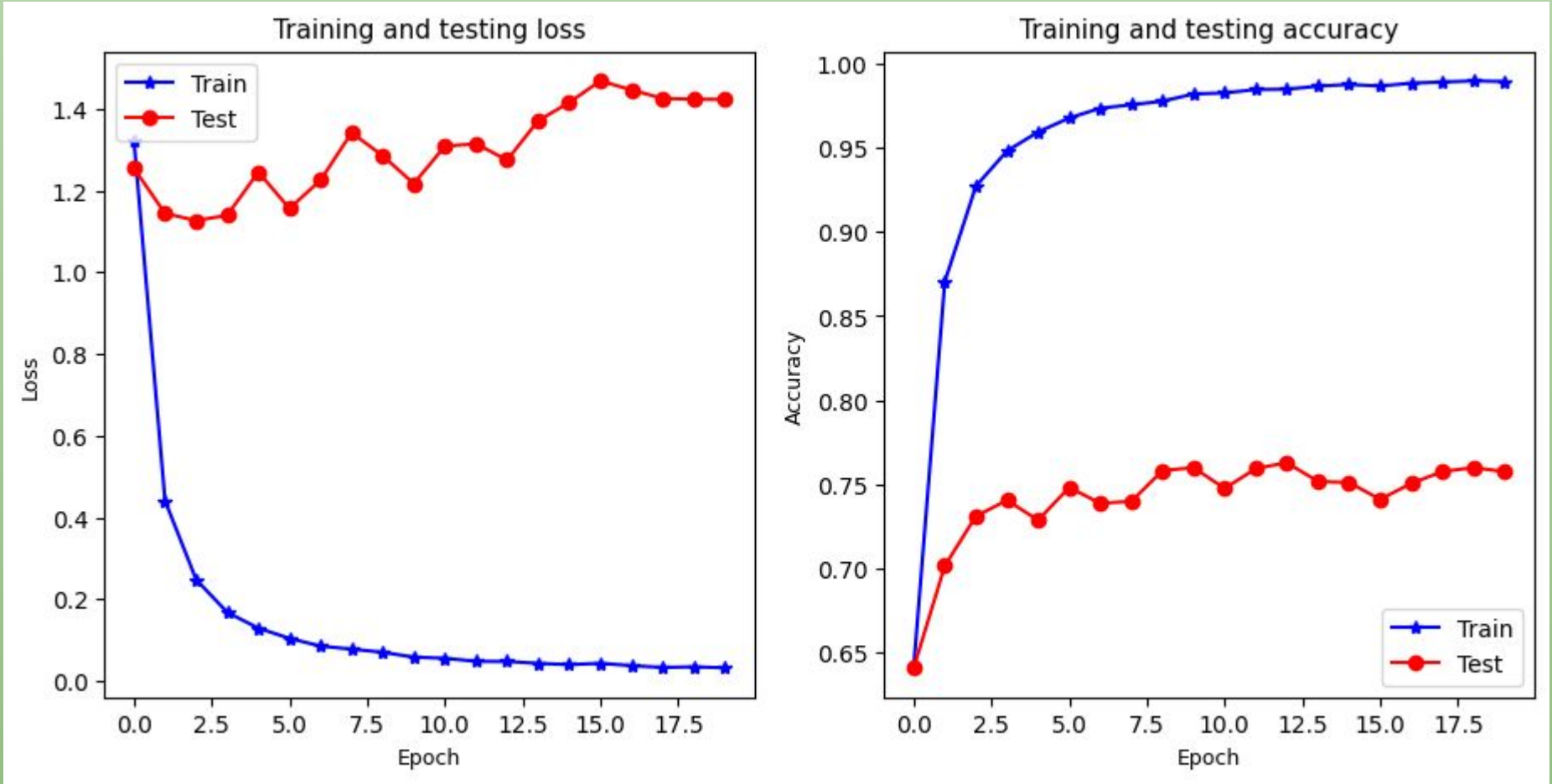
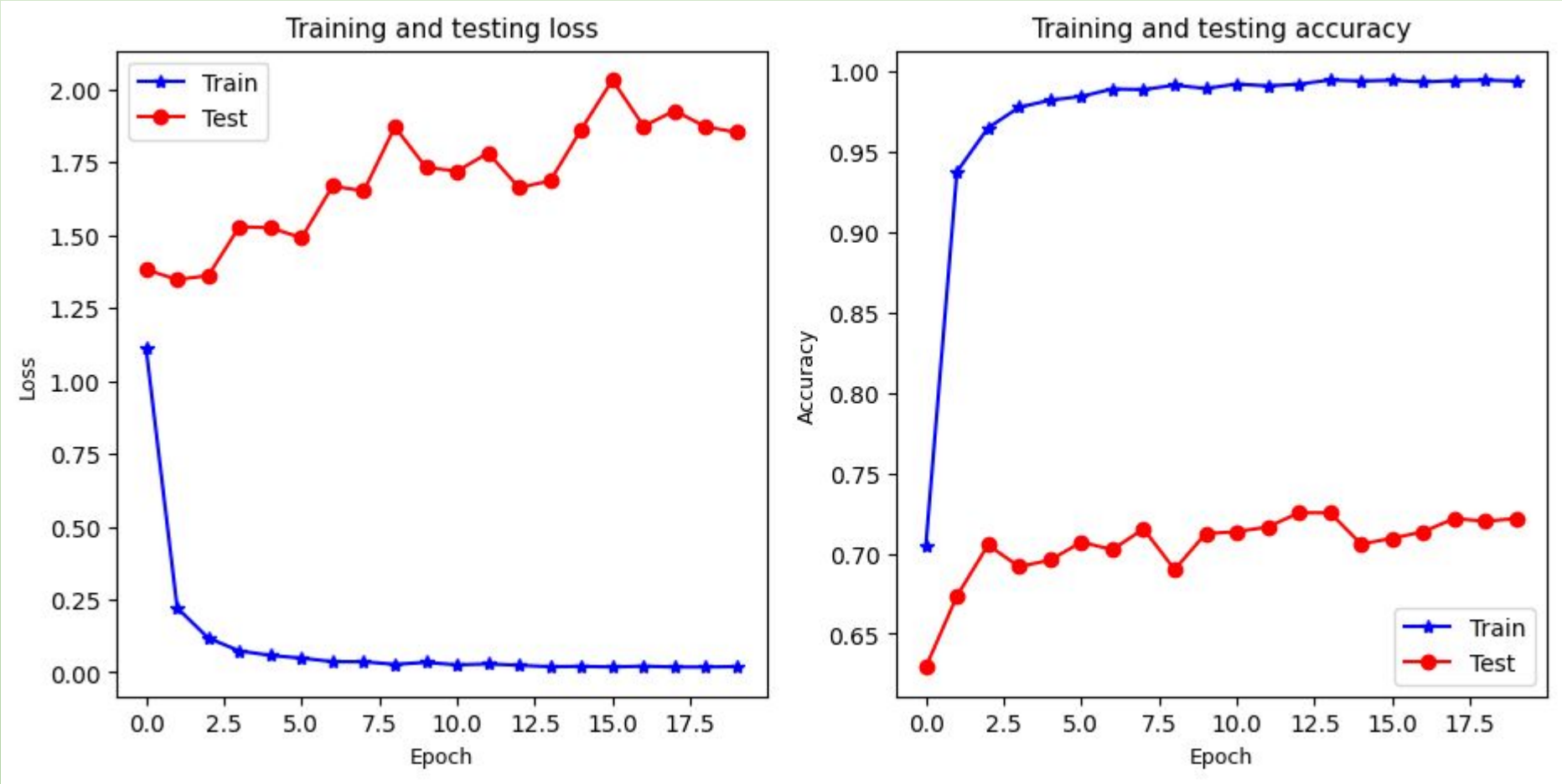
Было проведено 7 экспериментов. Первый эксперимент провалился полностью, модель mobilenetv2, настройка - feature extraction, не обучилась 😞



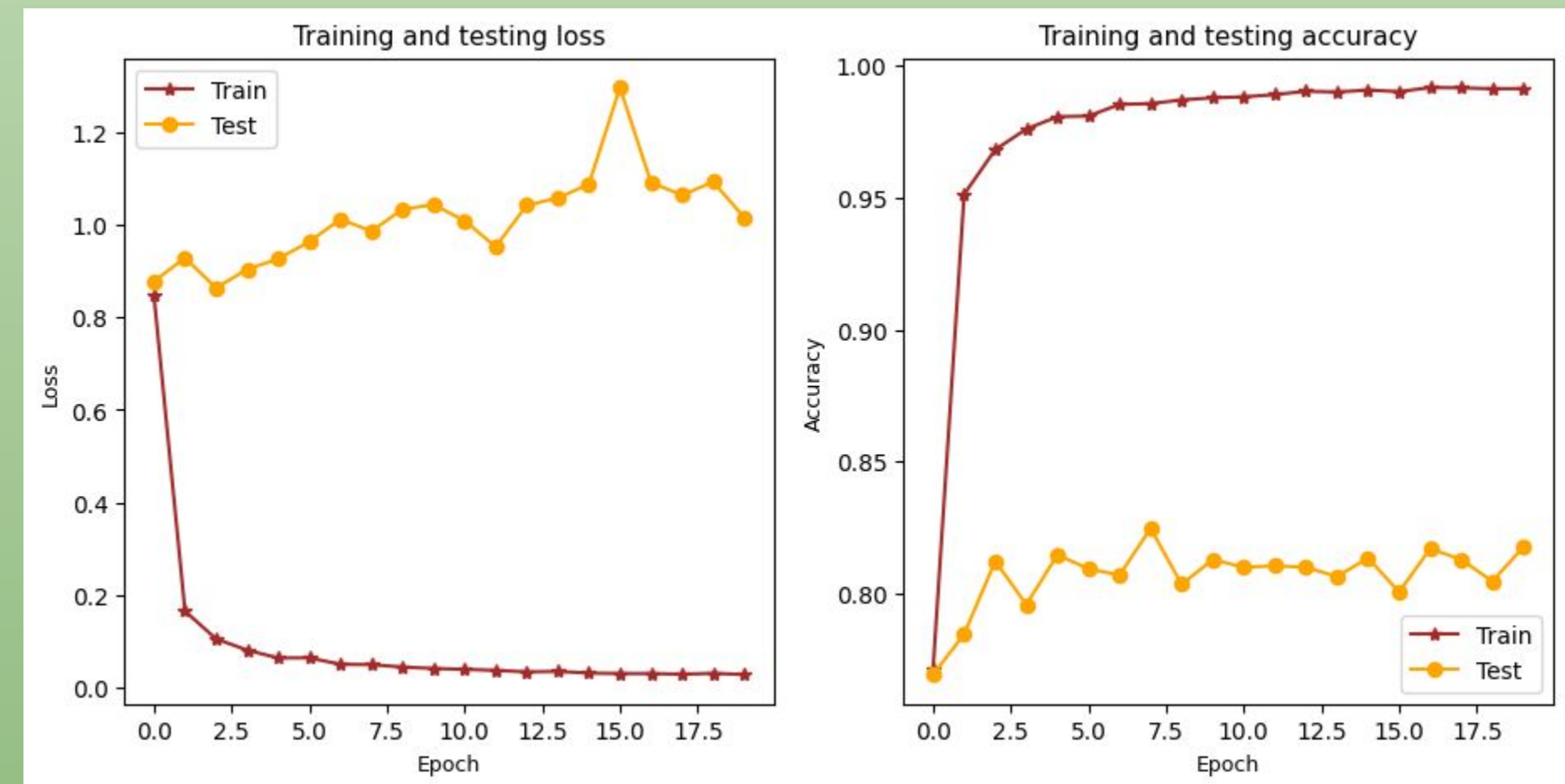
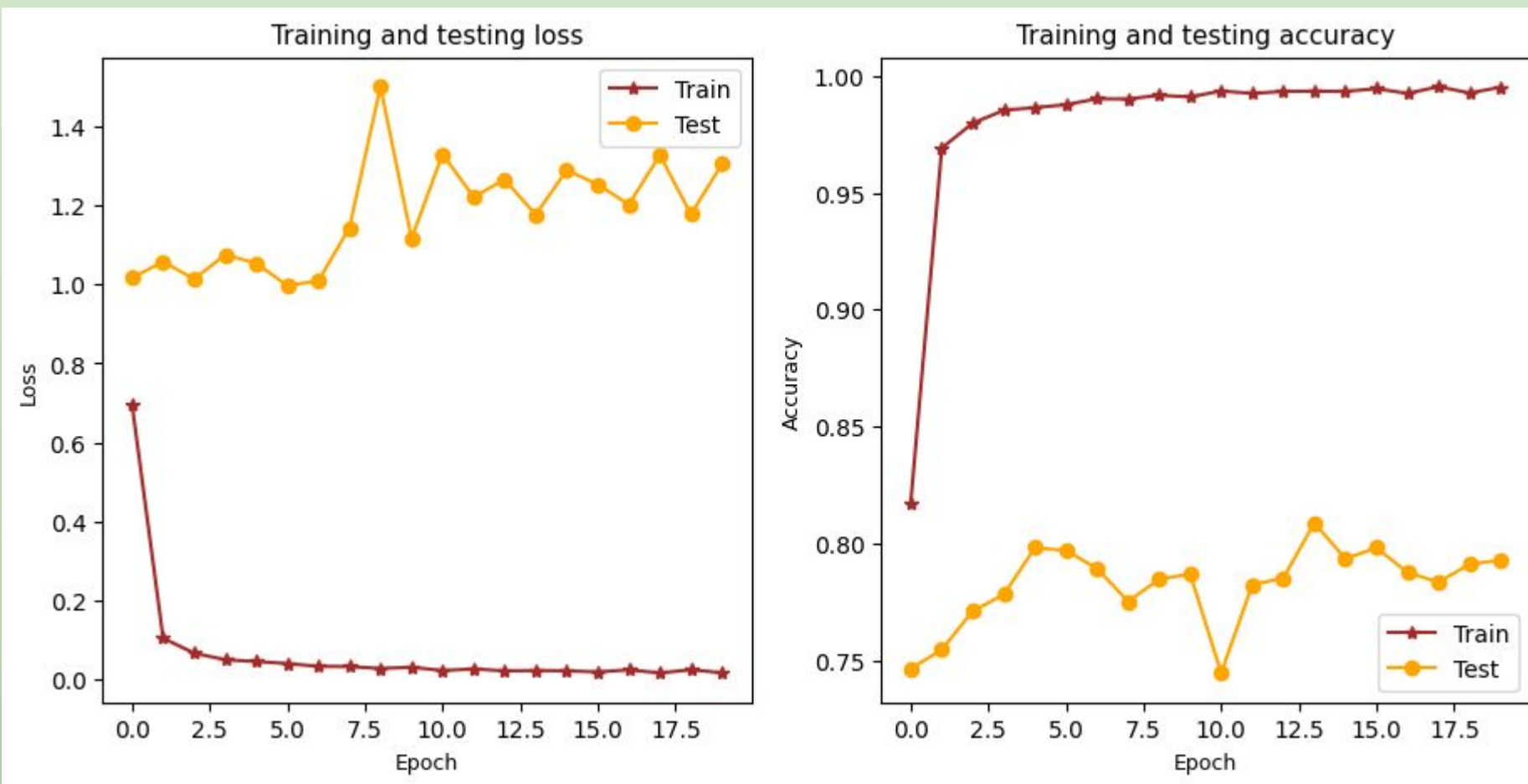
2 и 3 эксперименты, модель mobilenetv2, настройка fine-tuning без аугментации (слева) и с ней (справа).



4 и 5 эксперименты, модель mobilenetv3, настройка fine-tuning без аугментации (слева) и с ней (справа).



6 и 7 эксперименты, модель efficientnetv3_s, настройка fine-tuning без аугментации (слева) и с ней (справа).



Результаты экспериментов на тестовых данных

Модели	Потеря	Точность (%)
mobilenet_v2	1.790	72.29
mobilenet_v2_aug	1.329	78.12
mobilenet_v3_s	1.853	72.18
mobilenet_v3_s_aug	1.423	75.76
efficientnet_v2_s	1.305	79.29
efficientnet_v2_s_aug	1.016	81.76



Вывод: Цель этой работы достигнута ✓
Можно также в экспериментах применить:

Для устранения
дисбаланса:
умножить loss
редких классов
пропорциональ
но их редкости

Dropout для
устранения
переобучен
ия

Обернуть
Adam в какой-
нибудь
schedulers

Применить
другие виды
аугментации

Увеличить
количество
эпох



Э.Уорхол



СПАСИБО!

